

Data Structures & Algorithms

Hands-On Exercise Solutions

Exercise 1: Inventory Management System

Scenario

We're building the backend logic for a warehouse inventory system — something that needs to handle a large, constantly changing set of products without slowing down every time someone adds, updates, or removes stock.

1. Why data structures and algorithms matter here

A warehouse might be tracking anywhere from a few hundred to a few hundred thousand SKUs. If we just dumped everything into a plain list and searched through it one item at a time every time we needed to update a quantity, the system would get noticeably slower as inventory grows. That's the whole point of studying data structures: the *same* logical operation (find a product, update its stock) can cost wildly different amounts of time depending on how the data is organized underneath.

For an inventory system specifically, the operations that matter most are:

- **Lookup by ID** — happens constantly (every sale, every restock)

- **Insert new product** — happens whenever new stock arrives
- **Update quantity/price** — the most frequent operation of all
- **Delete a discontinued product** — rare, but still needs to be efficient

Picking the right structure means these day-to-day operations stay fast even as the catalog grows into the thousands.

2. Choosing the data structure

Two realistic options come up:

- **ArrayList** — good if we mostly iterate through everything (e.g., generating a full inventory report) and don't search by ID very often. But looking up a specific product means scanning from the front until we find it — $O(n)$ in the worst case.
- **HashMap** — since every product already has a unique productId, we can use that as the key. Lookup, insert, and delete all become close to $O(1)$ on average, since the hashmap jumps almost directly to the bucket where the product lives.

For a warehouse system where “find this product and update its quantity” is the most common operation by far, a `HashMap<String, Product>` keyed by productId is the better fit. I'll keep the discussion of both, but the implementation below uses the HashMap.

3. Implementation

```
import java.util.HashMap;

// The core entity we're storing
class Product {
    private String productId; private String
    productName; private int quantity;
    private double price;

    public Product(String productId, String productName, int quantity, double price) {
        this.productId = productId;
        this.productName = productName; this.quantity =
        quantity; this.price = price;
    }

    public String getProductId() { return productId; } public String
    getProductName() { return productName; } public int getQuantity() { return
    quantity; }
    public double getPrice() { return price; }

    public void setQuantity(int quantity) { this.quantity = quantity; } public void setPrice(double price) {
    this.price = price; }
```

```

@Override
public String toString() {
    return String.format("[%s] %s | qty: %d | price: %.2f",
        productId, productName, quantity, price);
}
}

class InventoryManager {
    // productId -> Product, gives us near O(1) access
    private HashMap<String, Product> inventory = new HashMap<>();

    public void addProduct(Product p) {
        if (inventory.containsKey(p.getProductId())) {
            System.out.println("Product " + p.getProductId() + " already exists. Use up
            return;
        }
        inventory.put(p.getProductId(), p);
    }

    public void updateProduct(String productId, int newQuantity, double newPrice) {
        Product p = inventory.get(productId);
        if (p == null) {
            System.out.println("No product found with ID " + productId);
            return;
        }
        p.setQuantity(newQuantity);
        p.setPrice(newPrice);
    }

    public void deleteProduct(String productId) {
        if (inventory.remove(productId) == null) {
            System.out.println("No product found with ID " + productId);
        }
    }

    public Product getProduct(String productId) {
        return inventory.get(productId);
    }

    public void printAll() {
        for (Product p : inventory.values()) {
            System.out.println(p);
        }
    }
}

public class InventoryDemo {

```

```

public static void main(String[] args) {
    InventoryManager manager = new InventoryManager();

    manager.addProduct(new Product("P001", "Wireless Mouse", 150, 499.00));
    manager.addProduct(new Product("P002", "Mechanical Keyboard", 80, 2199.00));
    manager.addProduct(new Product("P003", "USB-C Hub", 200, 899.00));

    manager.updateProduct("P002", 75, 2099.00);
    manager.deleteProduct("P003");

    manager.printAll();
}
}

```

4. Complexity analysis

Operation	ArrayList	HashMap
Add	$O(1)$ amortized (append)	$O(1)$ average
Search by ID	$O(n)$	$O(1)$ average
Update	$O(n)$ to find + $O(1)$ to update = $O(n)$	$O(1)$ average
Delete	$O(n)$ to find + $O(n)$ to shift = $O(n)$	$O(1)$ average

The HashMap wins on every operation that involves looking something up by ID, which is most of what an inventory system does. The tradeoff is that a HashMap doesn't preserve insertion order and uses somewhat more memory per entry (due to hashing overhead) — if ordered iteration or range queries by price were a frequent requirement, a TreeMap would be worth considering instead.

Optimization ideas: - Use LinkedHashMap instead of HashMap if we also need to preserve the order products were added, without giving up the $O(1)$ lookups. - If searches by name or category are also common, maintain a secondary index rather than scan-ning the whole inventory. - Batch updates can be done in bulk without triggering individual lookups each time.

Exercise 2: E-commerce Platform Search Function

Scenario

We need the search bar on an e-commerce site to feel instant, even when the product catalog is large.

1. Understanding asymptotic notation

Big O notation describes how the *runtime* (or memory use) of an algorithm grows as the input size grows — it deliberately ignores constant factors and hardware differences so we can compare algorithms on a level playing field. When we say an algorithm is $O(n)$, we mean the work grows roughly in a straight line with the number of items; $O(\log n)$ means the work barely grows at all even as input gets huge; $O(n^2)$ means it grows quadratically, which gets painful fast.

For a search operation, we usually talk about three cases:

- **Best case** — the item we're looking for happens to be found almost immediately (e.g., it's the very first element checked).
- **Average case** — a realistic estimate assuming the item could be anywhere with roughly equal probability.
- **Worst case** — the item isn't in the collection at all (or it's the very last one checked), so every element gets examined.

These matter because two algorithms can have the same worst case but behave very differently on typical inputs — which is exactly the situation with linear vs. binary search.

2. Setup

```
class Product {
    private String productId;
    private String productName;
    private String category;

    public Product(String productId, String productName, String category) {
        this.productId = productId;
        this.productName = productName;
        this.category = category;
    }

    public String getProductId() { return productId; }
    public String getProductName() { return productName; }
    public String getCategory() { return category; }

    @Override
    public String toString() {
        return productName + " (" + productId + ", " + category + ")";
    }
}
```

3. Implementation — linear search vs. binary search

```

import java.util.Arrays; import
java.util.Comparator;

class ProductSearch {

    // Works on any array, sorted or not — checks every element until a match public static Product
    linearSearch(Product[] products, String name) {
        for (Product p : products) {
            if (p.getProductName().equalsIgnoreCase(name)) { return p;
            }
        }
        return null;
    }

    // Requires the array to already be sorted by productName
    public static Product binarySearch(Product[] sortedProducts, String name) {
        int low = 0, high = sortedProducts.length - 1;

        while (low <= high) {
            int mid = low + (high - low) / 2;
            int cmp = sortedProducts[mid].compareToIgnoreCase(name);

            if (cmp == 0) {
                return sortedProducts[mid];
            } else if (cmp < 0) {
                low = mid + 1; // target is in the right half
            } else {
                high = mid - 1; // target is in the left half
            }
        }
        return null;
    }
}

public class SearchDemo {
    public static void main(String[] args) {
        Product[] products = {
            new Product("P010", "Bluetooth Speaker", "Audio"), new Product("P011",
            "Gaming Chair", "Furniture"), new Product("P012", "Laptop Stand",
            "Accessories"),
            new Product("P013", "Noise Cancelling Headphones", "Audio"), new Product("P014",
            "Standing Desk", "Furniture")
        };

        // Linear search doesn't care about order
        Product found = ProductSearch.linearSearch(products, "Laptop Stand");
    }
}

```

```
System.out.println("Linearsearchresult:"+found);

//Binarysearchneedsasorted-by-namearrayfirst Product[]
sorted = products.clone();
Arrays.sort(sorted,Comparator.comparing(Product::getProductName));
Productfound2=ProductSearch.binarySearch(sorted,"StandingDesk");
System.out.println("Binary search result: " + found2);
}
}
```

4. Analysis

	Linear Search	Binary Search
Best case	$O(1)$	$O(1)$
Average case	$O(n)$	$O(\log n)$
Worst case	$O(n)$	$O(\log n)$
Precondition	none	array must be sorted

Binary search wins hands down once the catalog grows large — searching a million products takes roughly 20 comparisons instead of up to a million. But that speed isn't free: it only works on sorted data, and keeping the data sorted has its own cost ($O(n \log n)$ to sort initially, and re-sorting or doing an $O(n)$ insertion whenever a new product is added, unless you use a self-balancing structure).

For the e-commerce search bar, the realistic answer is a bit more nuanced than “just use binary search”: most real search bars actually rely on **indexing structures** built for text search (like inverted indexes or trie-based prefix search) rather than binary search over a flat sorted array, especially once you factor in partial matches, typos, and multi-word queries. But for exact-match or ID-based lookups where the data is naturally sortable and doesn't change every second, binary search (or better, a sorted index maintained by a `TreeMap`) is the practical choice over plain linear scanning.

Exercise 3: Sorting Customer Orders

Scenario

Orders need to be sorted by totalPriceso the highest-value orders can be prioritized.

1. Understanding the sorting algorithms

- **Bubble Sort** — repeatedly walks through the list, swapping adjacent elements that are out of order. After each full pass, the largest unsorted element “bubbles” to its correct position. Simple to write, but slow — it does a lot of unnecessary comparisons.

- **Insertion Sort** — builds the sorted list one element at a time, taking each new element and inserting it into the correct position among the already-sorted portion. Efficient for small or nearly-sorted datasets.
- **Quick Sort** — picks a “pivot” element, partitions the array so smaller elements go left and larger ones go right, then recursively sorts each half. Very fast in practice because of good cache behavior and low overhead.
- **Merge Sort** — splits the array in half recursively until each piece has one element, then merges the pieces back together in sorted order. Consistent $O(n \log n)$ performance regardless of the input, at the cost of needing extra memory for merging.

2. Setup

```
class Order {
    private String orderId; private String
    customerName; private double
    totalPrice;

    public Order(String orderId, String customerName, double totalPrice) { this.orderId = orderId;
        this.customerName = customerName;
        this.totalPrice = totalPrice;
    }

    public double getTotalPrice() { return totalPrice; }

    @Override
    public String toString() {
        return String.format("%s | %s | ₹%.2f", orderId, customerName, totalPrice);
    }
}
```

3. Implementation — Bubble Sort and Quick Sort

```
class OrderSorter {

    // Bubble Sort: simple, but  $O(n^2)$  — fine for small order batches, not for large on public static void bubbleSort(Order[]
    orders) {
        int n = orders.length;
        for (int i = 0; i < n - 1; i++) {
            boolean swapped = false;
            for (int j = 0; j < n - 1 - i; j++) {
                if (orders[j].getTotalPrice() > orders[j + 1].getTotalPrice()) {
                    Order temp = orders[j]; orders[j] =
                    orders[j + 1]; orders[j + 1] = temp;
                }
            }
        }
    }
}
```

```

        swapped = true;
    }
}
// If nothing was swapped this pass, the array is already sorted if (!swapped) break;
}
}

// Quick Sort: divide-and-conquer, much better for larger datasets public static void quickSort(Order[]
orders, int low, int high) {
    if (low < high) {
        int pivotIndex = partition(orders, low, high); quickSort(orders, low,
            pivotIndex - 1); quickSort(orders, pivotIndex + 1, high);
    }
}

private static int partition(Order[] orders, int low, int high) {
    double pivot = orders[high].getTotalPrice(); int i = low - 1;

    for (int j = low; j < high; j++) {
        if (orders[j].getTotalPrice() <= pivot) {
            i++;
            Order temp = orders[i]; orders[i] =
                orders[j]; orders[j] = temp;
        }
    }
    Order temp = orders[i + 1]; orders[i + 1] =
        orders[high]; orders[high] = temp;

    return i + 1;
}

}

public class SortDemo {
    public static void main(String[] args) {
        Order[] orders1 = {
            new Order("O100", "Ravi", 2500.00),
            new Order("O101", "Meena", 750.00),
            new Order("O102", "Kiran", 4200.00), new
            Order("O103", "Divya", 1100.00)
        };

        Order[] orders2 = orders1.clone();
    }
}

```

```

OrderSorter.bubbleSort(orders1);
System.out.println("Sorted with Bubble Sort:");
for (Order o : orders1) System.out.println(o);

OrderSorter.quickSort(orders2, 0, orders2.length - 1);
System.out.println("\nSorted with Quick Sort:");
for (Order o : orders2) System.out.println(o);
    }
}

```

4. Analysis

	Bubble Sort	Quick Sort
Best case	$O(n)$ (already sorted, with the early-exit flag)	$O(n \log n)$
Average case	$O(n^2)$	$O(n \log n)$
Worst case	$O(n^2)$	$O(n^2)$ — rare, happens on already-sorted/reverse-sorted input with a poor pivot choice
Space	$O(1)$	$O(\log n)$ for the recursion stack
Stable?	Yes	No (relative order of equal elements can change)

Quick Sort is generally preferred because its *average-case* performance is $O(n \log n)$, which scales far better as the number of orders grows — sorting 10,000 orders with Bubble Sort could mean around 100 million comparisons, while Quick Sort would need roughly 130,000. Bubble Sort's only real advantages are that it's dead simple to write and reason about, and it's genuinely efficient if the data is already nearly sorted (which can happen if we're just inserting a handful of new orders into an already-sorted list — in that case Insertion Sort would actually be a better middle ground than either of these).

One caveat worth mentioning: Quick Sort's worst case ($O(n^2)$) does exist, and it shows up when the pivot choice is consistently bad. In production code this is usually avoided by picking a random pivot or using the median-of-three rule.

Exercise 4: Employee Management System

Scenario

We're storing and managing employee records for a company using a plain array.

1. How arrays are represented in memory

An array is a block of memory allocated as one contiguous chunk, where each element sits right next to the previous one. Because the size of each element is fixed and known ahead of time, the system can calculate the exact memory address of any element using simple arithmetic: $\text{address} = \text{base_address} + (\text{index} * \text{element_size})$. That's what makes indexed access — `array[i]` — $O(1)$: there's no searching involved, just a direct address calculation.

The advantage of this layout is speed of access and good cache locality. The disadvantage is rigidity: the size has to be fixed upfront, and inserting or removing something in the middle means shifting every element after it.

2. Setup

```
class Employee {
    private String employeeId; private
    String name; private String position;
    private double salary;

    public Employee(String employeeId, String name, String position, double salary) { this.employeeId = employeeId;
        this.name = name; this.position =
        position; this.salary = salary;
    }

    public String getEmployeeId() { return employeeId; }

    @Override
    public String toString() {
        return String.format("%s | %s | %s | ₹%.2f", employeeId, name, position, salary
    }
}
```

3. Implementation — array-based CRUD

```
class EmployeeArrayManager {
    private Employee[] employees;
    private int count;

    public EmployeeArrayManager(int capacity) {
        employees = new Employee[capacity];
        count = 0;
    }
}
```

```

    }

    public boolean add(Employee e) {
        if (count == employees.length) {
            System.out.println("Array is full — cannot add more employees."); return false;
        }
        employees[count] = e; count++;
        return true;
    }

    // Linear search since a plain array isn't indexed by ID public Employee
    search(String employeeId) {
        for (int i = 0; i < count; i++) {
            if (employees[i].getEmployeeId().equals(employeeId)) { return employees[i];
            }
        }
        return null;
    }

    public void traverse() {
        for (int i = 0; i < count; i++) {
            System.out.println(employees[i]);
        }
    }

    public boolean delete(String employeeId) { for (int i = 0; i <
        count; i++) {
        if (employees[i].getEmployeeId().equals(employeeId)) {
            // shift everything after this index one step left for (int j = i; j < count - 1; j++)
            {
                employees[j] = employees[j + 1];
            }
            employees[count - 1] = null;
            count--; return
            true;
        }
    }
    return false;
}

}

public class EmployeeDemo {
    public static void main(String[] args) {
        EmployeeArrayManager manager = new EmployeeArrayManager(10);
    }
}

```

```

manager.add(new Employee("E01", "Arjun Rao", "Analyst", 55000));
manager.add(new Employee("E02", "Priya Nair", "Engineer", 72000));
manager.add(new Employee("E03", "Sanjay Verma", "Manager", 95000));

manager.delete("E02");
manager.traverse();
}
}

```

4. Analysis

Operation	Time Complexity	Why
Add (at end)	$O(1)$	just place at <code>employees[count]</code> , assuming space is available
Search	$O(n)$	no ordering to exploit, must check each element
Traverse	$O(n)$	visiting every element is unavoidable
Delete	$O(n)$	find the element ($O(n)$) + shift remaining elements left ($O(n)$)

Limitations of arrays: - Fixed size — once full, adding more employees means creating a bigger array and copying everything over ($O(n)$). - Costly insertions/deletions in the middle, since elements have to shift to keep things contiguous. - No built-in fast lookup by a key like `employeeId` — you’re stuck with $O(n)$ search unless you sort it (enabling binary search) or pair it with a hash-based index.

When arrays still make sense: when the number of employees is fixed or rarely changes, when index-based access is the dominant use case (e.g., “give me employee #5”), or when memory predictability and cache performance matter more than flexibility. For a system where employees are hired and leave frequently, something like an `ArrayList` (which handles resizing for you) or a `HashMap<String, Employee>` (for $O(1)$ lookups by ID) would be a more practical real-world choice — but the plain array version above is exactly what the exercise asks for, and it’s a useful baseline to compare those richer structures against.

Exercise 5: Task Management System

Scenario

Tasks need to be added, deleted, and traversed efficiently — a good fit for exploring linked lists as an alternative to arrays.

1. Understanding linked lists

A linked list stores elements as separate objects (“nodes”) scattered anywhere in memory, where each node holds its data plus a reference (pointer) to the next node. Unlike an array, there’s no requirement for the elements to sit next to each other physically — the list is held together purely by those pointers.

- **Singly Linked List** — each node points only to the *next* node. You can walk forward through the list, but not backward.
- **Doubly Linked List** — each node points to both the *next* and the *previous* node. This costs a bit more memory per node but makes backward traversal and certain deletions faster, since you don’t need to walk from the head to find the node before the one you’re removing.

2. Setup

```
class Task {
    private String taskId;
    private String taskName;
    private String status;

    public Task(String taskId, String taskName, String status) {
        this.taskId = taskId;
        this.taskName = taskName;
        this.status = status;
    }

    public String getTaskId() { return taskId; }

    @Override
    public String toString() {
        return taskId + " | " + taskName + " | " + status;
    }
}
```

3. Implementation — singly linked list

```
class TaskNode {
    Task data;
    TaskNode next;
```

```

TaskNode(Task data) {
    this.data = data;
    this.next = null;
}
}

class TaskLinkedList { private
    TaskNode head; private int size;

    // Adds a new task to the end of the list public void add(Task
    task) {
        TaskNode newNode = new TaskNode(task); if (head ==
        null) {
            head = newNode;
        } else {
            TaskNode current = head;
            while (current.next != null) {
                current = current.next;
            }
            current.next = newNode;
        }
        size++;
    }

    public Task search(String taskId) {
        TaskNode current = head; while (current
        != null) {
            if (current.data.getTaskId().equals(taskId)) { return current.data;
            }
            current = current.next;
        }
        return null;
    }

    public void traverse() { TaskNode current =
        head; while (current != null) {
            System.out.println(current.data); current =
            current.next;
        }
    }

    public boolean delete(String taskId) { if (head == null)
        return false;

```



```

        // special case: deleting the head itself if
        (head.data.getTaskId().equals(taskId)) {
            head = head.next; size--;
            return true;
        }

        TaskNode current = head;
        while (current.next != null) {
            if (current.next.data.getTaskId().equals(taskId)) {
                current.next = current.next.next; // skip over the node being removed size--;
                return true;
            }
            current = current.next;
        }
        return false;
    }

    public int size() { return size; }
}

public class TaskDemo {
    public static void main(String[] args) {
        TaskLinkedList tasks = new TaskLinkedList();

        tasks.add(new Task("T01", "Design wireframes", "In Progress")); tasks.add(new Task("T02",
        "Set up database schema", "Pending")); tasks.add(new Task("T03", "Write unit tests",
        "Pending"));

        tasks.delete("T02");
        tasks.traverse();
    }
}

```

4. Analysis

Operation	Time Complexity	Notes
Add at end	O(n) (need to walk to the last node) — or O(1) if we keep a tail pointer	
Add at front	O(1)	just repoint head
Search	O(n)	must walk node by node, no random access
Traverse	O(n)	visiting every node

Operation	Time Complexity	Notes
Delete (given the ID)	$O(n)$	need to find the node first, then relink

Why linked lists beat arrays for this kind of dynamic data: - No fixed capacity — the list grows and shrinks one node at a time, with no need to allocate a bigger block and copy everything over. - Deleting or inserting in the *middle*, once you already have a reference to the right spot, is $O(1)$ — you’re just adjusting a couple of pointers, versus an array’s $O(n)$ element-shifting. - Memory is only used for tasks that actually exist, rather than reserving unused capacity upfront.

The tradeoff is that linked lists give up random access entirely (no `list.get(5)` in $O(1)$ the way arrays offer it) and use a bit more memory per element for the pointer overhead. For a task manager where tasks are constantly being added and removed but rarely accessed “by position,” that tradeoff is well worth it.

Exercise 6: Library Management System

Scenario

Users need to search for books by title, and we’re comparing two classic approaches: linear and binary search.

1. Linear search vs. binary search — the concepts

Linear search checks each item one at a time from the start until it finds a match or reaches the end. It doesn’t require the data to be in any particular order, which makes it the safe default when you can’t guarantee sorted input.

Binary search works only on sorted data. It repeatedly checks the middle element and, using the fact that the data is ordered, throws away half the remaining search space with each comparison — which is why it scales so much better than linear search as the dataset grows.

2. Setup

```
class Book {
    private String bookId;
    private String title;
    private String author;

    public Book(String bookId, String title, String author) {
        this.bookId = bookId;
        this.title = title;
        this.author = author;
    }
}
```

```

    }

    public String getTitle() { return title; }

    @Override
    public String toString() {
        return title + " by " + author + " (" + bookId + ") ";
    }
}

```

3. Implementation

```

import java.util.Arrays; import
java.util.Comparator;

class BookSearch {

    public static Book linearSearchByTitle(Book[] books, String title) { for (Book b : books) {
        if (b.getTitle().equalsIgnoreCase(title)) { return b;
        }
    }
    return null;
}

// Assumes books[] is already sorted by title
public static Book binarySearchByTitle(Book[] sortedBooks, String title) {
    int low = 0, high = sortedBooks.length - 1;

    while (low <= high) {
        int mid = low + (high - low) / 2;
        int cmp = sortedBooks[mid].getTitle().compareToIgnoreCase(title);

        if (cmp == 0) return sortedBooks[mid]; else if (cmp < 0)
            low = mid + 1;
        else high = mid - 1;
    }
    return null;
}

}

public class LibraryDemo {
    public static void main(String[] args) {
        Book[] books = {
            new Book("B01", "Clean Code", "Robert C. Martin"),

```

```

        new Book("B02", "Introduction to Algorithms", "Cormen et al."),
        new Book("B03", "The Pragmatic Programmer", "Hunt & Thomas"),
        new Book("B04", "Design Patterns", "Gang of Four")
    };

    Book found = BookSearch.linearSearchByTitle(books, "Design Patterns");
    System.out.println("Linear search found: " + found);

    Book[] sorted = books.clone();
    Arrays.sort(sorted, Comparator.comparing(Book::getTitle));
    Book found2 = BookSearch.binarySearchByTitle(sorted, "Clean Code");
    System.out.println("Binary search found: " + found2);
}
}

```

4. Analysis

	Linear Search	Binary Search
Time complexity	$O(n)$	$O(\log n)$
Needs sorted data	No	Yes
Extra cost	None	$O(n \log n)$ to sort first, if not already sorted

When to use which: - For a small library (a few hundred books) or a catalog that's rarely searched, linear search is perfectly fine — the difference in speed is negligible and it saves the trouble of keeping the data sorted. - For a large library catalog that's searched constantly and doesn't change every second, binary search is clearly worth it — the catalog can be sorted once and searched in logarithmic time from then on. - If the catalog changes frequently *and* needs fast search, the practical answer in real systems is usually a balanced tree (TreeMap) or a hash-based index, which avoid the need to fully re-sort after every insertion while keeping search fast.

Exercise 7: Financial Forecasting

Scenario

We want to predict a future value based on past growth rates using recursion.

1. Understanding recursion

Recursion is when a function solves a problem by calling itself on a smaller version of the same problem, until it reaches a “base case” simple enough to answer directly. It's a natural fit for problems that have a repeating, self-similar structure — compound

growth is a good example, because “the value after n years” is really just “the value after (n-1) years, grown by one more period.”

The general shape of a recursive solution is:

1. **Base case** — the smallest version of the problem, answered directly without further recursion
2. **Recursive case** — express the problem in terms of a smaller version of itself, and trust that the recursive call will correctly solve that smaller version.

2 & 3. Setup and implementation — recursive future value calculation

This models standard compound growth: $\text{futureValue} = \text{presentValue} * (1 + \text{growthRate})^{\text{years}}$, computed recursively instead of using `Math.pow` directly, since the exercise is about practicing recursion.

```
class FinancialForecaster {

    // Recursively computes future value given a present value, a per-period
    // growth rate (e.g., 0.08 for 8%), and the number of periods (years).
    public static double predictFutureValue(double presentValue, double growthRate, int
        // Base case: with 0 years left, the value hasn't grown any further if (years == 0) {
        return presentValue;
    }
    // Recursive case: grow it by one period, then recurse on the rest
    return predictFutureValue(presentValue * (1 + growthRate), growthRate, years -

}

// Memoized version — avoids recomputing the same (value, years) combination
// repeatedly, useful if predictFutureValue gets called many times with
// overlapping inputs (e.g., forecasting multiple scenarios).
public static double predictFutureValueMemoized(double presentValue, double growthR
    int years, java.util.Map<Integer,

    if (years == 0) return presentValue;
    if (cache.containsKey(years)) return cache.get(years) * (presentValue / 1.0);
    // Note: caching here is illustrative — see the analysis below for why
    // this particular problem doesn't actually need memoization in practice.
    double result = predictFutureValueMemoized(presentValue * (1 + growthRate), gro
        years - 1, cache);
    cache.put(years, result); return result;
}

}

public class ForecastDemo {
    public static void main(String[] args) {
        double startingValue = 100000.0; // e.g., current portfolio value in rupees
        double annualGrowthRate = 0.08; // 8% growth per year
    }
}
```

```

int yearsToForecast = 5;

double forecast = FinancialForecaster.predictFutureValue(startingValue, annualGrowthRate, yearsToForecast);
System.out.printf("Projected value after %d years: ₹%.2f%n", yearsToForecast, forecast);
}
}

```

4. Analysis

Time complexity: $O(n)$, where n is the number of years/periods. Each recursive call does a constant amount of work (one multiplication) and makes exactly one further recursive call, so the total work scales linearly with the number of periods.

Space complexity: $O(n)$ as well — this is the part that's easy to overlook. Every recursive call adds a new frame to the call stack, and none of those frames are removed until the base case is hit and the calls start returning. For a forecast over, say, 30 years, that's 30 stack frames sitting in memory at once — not a problem at this scale, but it's worth knowing that a *very* long forecast horizon (thousands of periods) could risk a stack overflow with naive recursion.

Optimizing it: - **Tail-call style / iterative rewrite** — since each call does trivial work and immediately returns the result of the next call, it can be rewritten as a simple loop that uses $O(1)$ space instead of $O(n)$:
 java public static double predictFutureValueIterative(double presentValue, double growthRate, int years) { double value = presentValue; for (int i = 0; i < years; i++) { value *= (1 + growthRate); } return value; }
 Java doesn't optimize tail calls automatically the way some functional languages do, so for a genuinely large years value, the iterative version is the safer choice in practice. - **Closed-form formula** — since this is just compound growth, $\text{presentValue} * \text{Math.pow}(1 + \text{growthRate}, \text{years})$ computes the exact same result in $O(\log n)$ time (thanks to fast exponentiation under the hood) without any recursion or looping at all. - **Memoization** only helps when the *same* sub-computation gets requested repeatedly across multiple calls. For a single one-off forecast like this, memoization adds bookkeeping overhead without any real benefit — it's a good instinct to reach for with recursion, but it's the wrong tool here since there's no overlapping sub-problem to cache.